DCN V2：改进的深度交叉网络及面向大规模网页排序系统的实践经

若曦·王，拉克什·希万纳，德里克·Z·程，萨加尔·贾恩，董林，李灿·洪，埃德·H·奇 谷歌公司 {ruoxi, rakeshshivanna, zcheng, sagarj, dongl, lichan, edchi}@google.com

0202 tcO 02]RI·sc[2v53531·80002:viXra

摘要

学习有效的特征交叉是构建推荐系统的关键。然而，稀疏且庞大的特征空间需要穷举搜索来识别有效的交叉。Deep & Cross Network（DCN）被提出用于自动且高效地学习有界阶的预测性特征交互。遗憾的是，在服务于具有数十亿训练样本的网络级流量的模型中，DCN在其交叉网络中在学习更具预测性的特征交互方面表现出有限的表达能力。尽管已经取得了显著的研究进展，但许多生产中的深度学习模型仍然依赖传统的前馈神经网络来低效地学习特征交叉。

鉴于DCN及现有特征交互学习方法的优缺点，我们提出了一个改进框架DCN-V2，以使DCN在大规模工业环境中更加实用。在进行了广泛的超参数搜索和模型调优的全面实验研究中，我们观察到DCN-V2方法在流行的基准数据集上优于所有最先进的算法。改进后的DCN-V2在特征交互学习方面更具表现力，同时保持成本效益，尤其是与低秩架构混合使用时。DCN-V2简单易用，可以轻松作为构建模块采用，并已在Google的许多网络规模学习排序系统中实现了显著的离线准确性和在线业务指标提升。

1 引言

学习排序（LTR）[4, 27]一直是现代机器学习和深度学习中最重要的问题之一。它在搜索、推荐系统[17, 39, 41]和计算广告[2, 3]中有着广泛的应用。在学习排序模型的关键组件中，学习有效的特征交叉持续吸引着学术界[26, 35, 46]和工业界[1, 6, 13, 34, 50]的大量关注。

有效的特征交叉对于许多模型的成功至关重要。它们提供了超越单个特征的额外交互信息。例如，“国家”和“语言”的组合比其中任何一个都更具信息量。在线性模型时代，机器学习从业者依赖手动识别此类特征交叉 [43] 来增加模型的表达能力。不幸的是，这涉及一个组合搜索空间，在数据主要为类别型的网络规模应用中，该空间既大又稀疏。在这种设定下进行搜索是穷举式的，通常需要领域专业知识，并且使模型更难泛化。

后来，嵌入技术被广泛采用，用于将特征从高维稀疏向量投影到维度低得多的稠密向量。因子分解机（FMs）[36, 37]利用嵌入技术并构建成对通过两个潜在向量的内积实现特征交互。与线性模型中的传统特征交叉相比，FM带来了更强的泛化能力。

在过去十年中，随着计算能力的增强和数据规模的扩大，工业界的LTR模型逐渐从线性模型和基于FM的模型迁移到深度神经网络（DNN）。这全面显著提升了搜索和推荐系统的模型性能[6, 13, 50]。人们通常认为DNN是通用函数逼近器，有可能学习各种特征交互[31, 47, 49]。然而，近期研究[1, 50]发现，DNN即使只是近似建模二阶或三阶特征交叉，效率也很低。

为了更准确地捕捉有效的特征交叉，一种常见的补救方法是通过更宽或更深的网络进一步增加模型容量。这自然形成了一把双刃剑：我们在提升模型性能的同时，也使模型的 serving 变得慢得多。在许多生产环境中，这些模型需要处理极高的 QPS，因此对实时推理有非常严格的延迟要求。可能的情况是，serving 系统已经被推到极限，无法承受更大的模型。此外，更深的模型往往会引入可训练性问题，使模型更难训练。

这揭示了设计一个能够高效且有效地学习预测性特征交互的模型的关键需求，尤其是在处理来自数十亿用户的实时流量的资源受限环境中。许多近期工作[1, 6, 13, 26, 34, 35, 46, 50]试图应对这一挑战。共同的主题是利用从DNN中学习到的那些隐式高阶交叉，以及在线性模型中被发现有效的显式且有界阶的特征交叉。隐式交叉意味着交互通过端到端函数学习，而没有任何显式公式对这种交叉建模。另一方面，显式交叉由具有可控交互阶数的显式公式建模。我们将在第2节中详细讨论这些模型。

其中，Deep & Cross Network (DCN) [50] 有效且优雅，然而，在大规模工业系统中将 DCN 生产化面临许多挑战。其交叉网络的表达能力有限。交叉网络所再现的多项式类仅由 O(输入大小) 参数来表征，这在很大程度上限制了其对随机交叉模式建模的灵活性。此外，交叉网络与 DNN 之间分配的容量不平衡。当将 DCN 应用于大规模生产数据时，这一差距显著增加。绝大部分参数将被用于学习 DNN 中的隐式交叉。

在本文中，我们提出了一个新模型DCN-V2，它改进了原始的DCN模型。我们已经成功地将DCN-V2部署在Google的多个学习排序系统中，在离线模型准确性和在线业务方面都取得了显著收益。

指标。DCN-V2首先通过交叉层学习输入（通常是嵌入层）的显式特征交互，然后与深度网络结合以学习互补的隐式交互。DCN-V2的核心是交叉层，它继承了DCN中交叉网络的简单结构，但在学习显式且有界阶的交叉特征方面显著更具表现力。该论文研究了以点击作为正标签的数据集，然而DCN-V2与标签无关，可以应用于任何学习排序系统。该论文的主要贡献有五个方面：

- 我们提出了一种新颖的模型——DCN-V2——来学习有效的显式和隐式特征交叉。与现有方法相比，我们的模型更具表达力，同时保持高效和简单。
- 观察到DCN-V2中所学矩阵的低秩特性，我们提出利用低秩技术来近似子空间中的特征交叉，以实现更好的性能和延迟权衡。此外，我们提出了一种基于专家混合架构[19, 45]的技术，进一步将矩阵分解为多个更小的子空间。这些子空间随后通过门控机制进行聚合。
- 我们进行并提供了一项使用合成数据集的广泛研究，该研究证明了传统的基于ReLU的神经网络在学习高阶特征交叉方面的低效性。
- 通过全面的实验分析，我们证明了我们提出的DCN-V2模型在Criteo和MovieLen-1M基准数据集上显著优于SOTA算法
- 我们提供了一个案例研究，并分享了在大规模工业排名系统中将DCN-V2生产化的经验教训，这带来了显著的离线和在线收益。

本文组织如下。第2节总结相关工作。第3节描述我们提出的模型架构DCN-V2及其内存高效版本。第4节分析DCN-V2。第5节提出若干研究问题，这些问题通过第6节在合成数据集和第7节在公开数据集上的全面实验予以回答。第8节描述将DCN-V2在生产环境中部署于一个网络规模推荐系统的过程。

2 相关工作

近期特征交互学习工作的核心思想是利用显式和隐式（来自DNN）的特征交叉。为了建模显式交叉，大多数近期工作引入了在DNN中效率低下的乘法操作（$x_1 \times x_2$），并设计了一个函数$f(\mathrm{x}_1, \mathrm{x}_2)$来高效且显式地建模特征$\mathrm{x}_1$和$\mathrm{x}_2$之间的成对交互。我们根据它们如何组合显式和隐式组件来组织这些工作。

并行结构。有一项工作联合训练两个并行网络，其灵感来自宽深模型[6]，其中宽组件将原始特征的交叉作为输入；而深组件是一个DNN模型。然而，为宽组件选择交叉特征又回到了线性模型的特征工程问题。尽管如此，宽深模型启发了许多工作采用这种并行架构，并对宽组件进行改进。

DeepFM [13] 通过采用 FM 模型，在 wide 组件中自动化了特征交互学习。DCN [50] 引入了一个交叉网络，该网络学习显式且有界阶的特征自动且高效地进行交互。xDeepFM [26] 通过生成多个特征图来增强 DCN 的表达能力，每个特征图编码当前层级与输入层级特征之间的所有两两交互。此外，它还将每个特征嵌入 x_i 视为一个单元，而不是将每个元素 x_i 视为一个单元。不幸的是，其计算成本显著较高（是参数数量的 10 倍），使其在工业规模应用中不切实际。此外，DeepFM 和 xDeepFM 都要求所有特征嵌入具有相同的大小，这又是在应用于词汇量大小（类别特征的大小）从 $O(10)$ 到数百万不等的工业数据时的另一个限制。AutoInt [46] 利用带有残差连接的多头自注意力机制。InterHAt [25] 进一步采用了分层注意力机制。

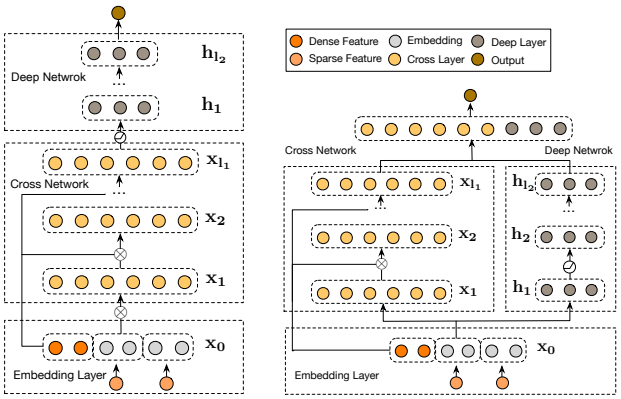
堆叠结构。另一条工作路线在嵌入层与DNN模型之间引入了一个交互层——该层显式地创建特征交叉。这个交互层在早期阶段捕获特征交互，并促进后续隐藏层的学习。基于乘积的神经网络（PNN）[35]引入了内积（IPNN）和外积（OPNN）乘积层作为成对交互层。OPNN的一个缺点在于其高计算成本。神经FM（NFM）[16]通过用Hadamard积替换内积来扩展FM；DLRM [34]遵循FM通过内积计算特征交叉；这些模型最多只能创建二阶显式交叉。AFN [7]将特征变换到对数空间，并自适应地学习任意阶特征交互。与DeepFM和xDeepFM类似，它们只接受相同大小的嵌入。

尽管已经取得了许多进展，我们的综合实验（第7节）表明，DCN仍然是一个强大的基线。我们将此归因于其简单的结构，这有助于优化。然而，如前所述，其有限的表达能力阻止了它在网络规模系统中学习更有效的特征交叉。在下文中，我们提出了一种新的架构，它继承了DCN的简单结构，同时增加了其表达能力。

3 提出的架构：DCN-V2

本节介绍了一种新颖的模型架构——DCN-V2——用于学习显式和隐式特征交互。DCN-V2从嵌入层开始，随后是一个包含多个交叉层的交叉网络，用于建模显式特征交互，然后与一个建模隐式特征交互的深度网络相结合。DCN-V2中所做的改进对于

将DCN应用于高度优化的生产系统实践中。DCN-V2显著提升了DCN[50]在对网络规模生产数据中复杂显式交叉项建模时的表达能力，同时保持了其优雅的公式以便于部署。DCN-V2所建模的函数类严格包含DCN所建模的函数类。整体模型架构如图1所示，交叉网络与深度网络有两种结合方式：（1）堆叠式和（2）并行式。此外，观察到交叉层的低秩特性，我们提出利用低秩交叉层的混合来实现模型性能与效率之间更健康的权衡。



(a) Stacked **(b) Parallel**

图1：DCN-V2的可视化。⊗ 表示式(1)中的交叉操作，即 $x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$。

3.1 嵌入层

嵌入层将类别（稀疏）特征和稠密特征的组合作为输入，并输出 $x_0 \in \mathbb{R}^d$。对于第 i 个类别特征，我们通过 $x_{embed,i} = W_{embed,i} e_i$ 将其从高维稀疏空间投影到低维稠密空间，其中 $e_i \in \{0, 1\}^{v_i}$；$W \in \mathbb{R}^{e_i \times v_i}$ 是学习到的投影矩阵；$x_{embed,i} \in \mathbb{R}^{e_i}$ 是稠密嵌入向量；v_i 和 e_i 分别表示词表大小和嵌入维度。对于多值特征，我们使用嵌入向量的均值作为最终向量。

输出是所有嵌入向量与归一化稠密特征的拼接：$x_0 = [x_{embed,1}; \ldots; x_{embed,n}; x_{dense}]$。

与许多相关作品[13, 16, 26, 34, 35, 46]需要$e_i = e_j \ \forall i, j$不同，我们的模型接受任意嵌入大小。这对于工业推荐系统尤为重要，因为其词汇量从$O(10)$到$O(10^5)$不等。此外，我们的模型并不局限于上述嵌入方法；任何其他嵌入技术，如哈希，都可以采用。

3.2 交叉网络

DCN-V2 的核心在于创建显式特征交叉的交叉层。式 (1) 展示了第 $(l+1)$ 个交叉层。

$$x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l \tag{1}$$

其中 $x_0 \in \mathbb{R}^d$ 是包含原始一阶特征的基础层，通常设置为嵌入（输入）层。$x_l, x_{l+1} \in \mathbb{R}^d$ 分别表示第 $(l+1)$ 个交叉层的输入和输出。$W_l \in \mathbb{R}^{d \times d}$ 和 $b_l \in \mathbb{R}^d$ 是学习到的权重矩阵和偏置向量。图2展示了一个单独的交叉层如何运作。

对于一个l层的交叉网络，最高多项式阶数为$l+1$，且该网络包含所有直至最高阶的特征交叉项。请参阅第4.1节，从按位和按特征两个角度进行详细分析。当$W = 1 \times w^\top$时，其中1表示全1向量，DCN-V2退化为DCN。

交叉层只能重现有界次数的多项式函数类；任何其他复杂的函数空间只能

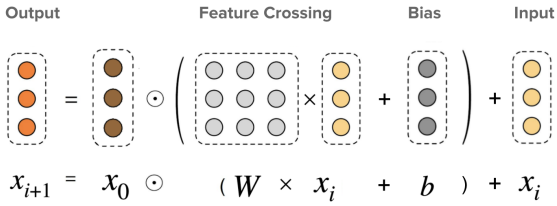


Figure 2: Visualization of a cross layer.

可以被近似1。因此，我们引入一个深度网络来补充对数据中固有分布的建模。

3.3 深度网络

第l个深层的公式由$h_{l+1} = f(W_l h_l + b_l)$给出，其中$h_l \in \mathbb{R}^{d_l}$, $h_{l+1} \in \mathbb{R}^{d_{l+1}}$分别是第$l$个深层的输入和输出；$W_l \in \mathbb{R}^{d_l \times d_{l+1}}$是权重矩阵，$b_l \in \mathbb{R}^{d_{l+1}}$是偏置向量；$f(\cdot)$是逐元素激活函数，我们将其设置为ReLU；任何其他激活函数也适用。

3.4 深度与交叉组合

我们寻求结合交叉网络和深度网络的结构。近期文献采用了两种结构：堆叠式和并行式。在实践中，我们发现哪种架构效果更好取决于数据。因此，我们同时给出这两种结构：

堆叠结构（图1a）：输入x_0被馈送到交叉网络，随后是深度网络，最终层由$x_{final} = h_{L_d}$, $h_0 = x_{L_c}$给出，其将数据建模为$f_{deep} \circ f_{cross}$。

并行结构（图1b）：输入x_0被并行馈送到交叉网络和深度网络；然后，输出x_{L_c}和h_{L_d}被拼接以创建最终输出层$x_{final} = [x_{L_c}; h_{L_d}]$。该结构将数据建模为$f_{cross} + f_{deep}$。

最后，预测值 $\hat{y}_i$ 计算为：$\hat{y}_i = \sigma(w^\top x_{final})$，其中 w_{logit} 是 logit 的权重向量，且 $\sigma(x) = 1/(1 + \exp(-x))$。对于最终损失，我们使用 Log Loss，它通常用于学习排序系统，尤其是带有二元标签（例如点击）的情况。请注意，DCN-V2 本身与预测任务和损失函数均无关。

$$\text{loss} = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i) + \lambda \sum_l \|W_l\|_2^2,$$

其中 $\hat{y}_i$ 是预测值；y_i 是真实标签；N 是输入总数；λ 是 L_2 正则化参数。

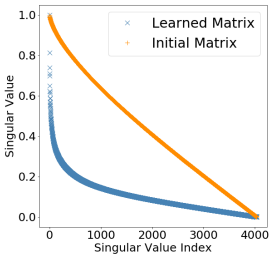
3.5 低成本低秩DCN混合

在实际生产模型中，模型容量往往受到有限的服务资源和严格的延迟要求的约束。通常情况下，我们不得不寻求在保持准确性的同时降低成本的方法。低秩技术[12]被广泛使用[5, 9, 14, 20, 51, 52]以降低计算成本。它通过两个高瘦矩阵$U, V \in \mathbb{R}^{d \times r}$来近似一个稠密矩阵$M \in \mathbb{R}^{d \times d}$。当$r \leq d/2$时，成本将会降低。然而，

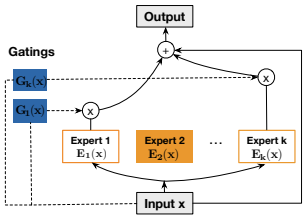
[1]Any function with certain smoothness assumptions can be well-approximated by polynomials. In fact, we've observed in our experiments that cross network alone was able to achieve similar performance as traditional deep networks.

当矩阵显示出奇异值的较大差距或快速谱衰减时，它们最为有效。在许多情况下，我们确实观察到学习到的矩阵在实践中数值上是低秩的。

图3a展示了来自生产模型的DCN-V2（见式(1)）中所学矩阵W的奇异值衰减模式。与初始矩阵相比，所学矩阵呈现出快得多的谱衰减模式。我们将容差为T的数值秩R_T定义为$\text{argmin}_k(\sigma_k < T \cdot \sigma_1)$，其中$\sigma_1 \geq \sigma_2 \geq, \ldots, \geq \sigma_n$为奇异值。那么，$R_T$意味着直至容差$T$的大部分质量被保留在前$k$个奇异值中。在机器学习和深度学习领域，模型在相当高的容差$_T$2下仍能工作得异常出色。



(a) Singular Values



(b) Mixture of Low-rank Experts

图3：左：学习到的DCN-V2权重矩阵的奇异值衰减。奇异值被归一化，$1 = \sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_k$。+ 表示随机初始化的截断正态矩阵；× 表示最终学习到的矩阵。右：低秩交叉层混合的可视化。

因此，对 W 施加低秩结构是很有动机的。式 (2) 展示了所得的 (l + 第 1) 个低秩交叉层

$$\mathrm{x}_{l+1} = \mathrm{x}_0 \odot \left(U_l\left(V_l^\top \mathrm{x}_i\right) + \mathrm{b}_l\right) + \mathrm{x}_i \tag{2}$$

其中 $U_l, V_l \in \mathbb{R}^{d\times r}$ 和 $r \ll d$。公式 (2) 有两种解释：1) 我们在子空间中学习特征交叉；2) 我们将输入 x 投影到低维的 $\mathbb{R}^r$，然后再将其投影回 $\mathbb{R}^d$。这两种解释启发了以下两种模型改进。

解释1启发我们采用混合专家（MoE）[10, 19, 30, 45]的思想。基于MoE的模型由两个部分组成：专家（通常是一个小型网络）和门控（输入的函数）。在我们的案例中，我们不依赖单一专家（公式(2)）来学习特征交叉，而是利用多个这样的专家，每个专家在不同的子空间中学习特征交互，并使用依赖于输入x的门控机制自适应地组合学习到的交叉。由此产生的低秩交叉层混合公式如公式(3)所示，并在图3b中描绘。

$$\begin{aligned} \mathrm{x}_{l+1} &= \sum_{i=1}^{K} G_i(\mathrm{x}_l) E_i(\mathrm{x}_l) + \mathrm{x}_l \\ E_i(\mathrm{x}_l) &= \mathrm{x}_0 \odot \left(U_l^i\left(V_l^{i\top} \mathrm{x}_l\right) + \mathrm{b}_l\right) \end{aligned} \tag{3}$$

[2]This is very different from the filed of scientific computing (*e.g.*, solving linear systems), where the approximation accuracy need to be very high. For problems such as CTR prediction, some errors could introduce regularization effect to the model.

其中 K 是专家数量；$G_i(\cdot): \mathbb{R}^d \mapsto \mathbb{R}$ 是门控函数，常见为 sigmoid 或 softmax；$E_i(\cdot): \mathbb{R}^d \mapsto \mathbb{R}^d$ 是学习特征交叉的第 $_i$ 个专家。$G(\cdot)$ 为输入 x 动态加权每个专家，当 $G(\cdot) \equiv 1$ 时，式 (3) 退化为式 (2)。

解释2启发我们利用投影空间的低维特性。我们不在维度d'到d ($d' \ll d$)之间立即投影回去，而是在投影空间中进一步应用非线性变换来细化表示[11]。

$$E_i(\mathrm{x}_l) = \mathrm{x}_0 \odot \left(U_l^i \cdot g\left(C_l^i \cdot g\left(V_l^{i\top} \mathrm{x}_l\right)\right) + \mathrm{b}_l\right) \tag{4}$$

其中 $g(\cdot)$ 表示任意非线性激活函数。

讨论。本节旨在有效利用固定的内存/时间预算来学习有意义的特征交叉。从公式(1)至(4)可以看出，在固定参数数量的假设下，每个公式都代表一个严格更大的函数类。

与许多在训练后进行压缩的模型压缩技术不同，我们的模型在训练之前就施加结构先验，并与其他参数联合学习相关参数。因此，交叉层是非线性系统$f(\mathrm{x}) = \left(f_k(W_k) \circ \cdots \circ f_1(W_1)\right)(\mathrm{x})$的组成部分，其中$(f_{i+1} \circ f_i)(\cdot) \coloneqq f_{i+1}(f_i(\cdot))$。因此，整个系统的训练动态可能会受到影响，观察全局统计量（如$f(\mathrm{x})$的雅可比矩阵和海森矩阵）如何受到影响将会很有趣。我们将此类研究留待未来工作。

3.6 复杂度分析

令 d 表示嵌入大小，L_c 表示交叉层数，K 表示低秩 DCN 专家的数量。此外，为简单起见，我们假设每个专家具有相同的较小维度 r (秩的上界)。交叉网络的时间复杂度和空间复杂度为 $O(d^2 L_c)$，而对于低秩 DCN 的混合（DCN-Mix），当 $rK \ll d$ 且 $O(2drKL_c)$ 时它是高效的。

4 模型分析

本节从多项式逼近的角度分析DCN-V2，并将其与相关工作联系起来。我们采用[50]中的符号表示。

符号说明。设嵌入向量 $\mathrm{x} = [\mathrm{x}_1; \mathrm{x}_2; \ldots; \mathrm{x}_k] = [x_1, x_2, \ldots, x_d] \in \mathbb{R}^d$ 为列向量，其中 $\mathrm{x}_i \in \mathbb{R}^{e_i}$ 表示第 i 个特征嵌入，x_i 表示 x 中的第 i 个元素。设多指标 $\boldsymbol{\alpha} = [\alpha_1, \cdots, \alpha_d] \in \mathbb{N}^d$ 和 $|\boldsymbol{\alpha}| = \sum_{i=1}^d \alpha_i$。$C_a^b \coloneqq \{\mathrm{y} \in \{1, \cdots, a\}^b \mid \forall i < j, y_i > y_j\}$。设 1 为全 1 向量，$I$ 为单位矩阵。我们用大写字母表示矩阵，粗体小写字母表示向量，普通小写字母表示标量。

4.1 多项式逼近

我们从多项式逼近的两个角度分析DCN-V2——1）将每个元素（位）x_i视为一个单元，并分析元素之间的交互（定理4.1）；2）将每个特征嵌入x_i视为一个单元，仅分析特征层面的交互（定理4.2）（证明见附录）。

定理 4.1（按位）。假设 l 层交叉网络的输入为 $\mathrm{x} \in \mathbb{R}^d$，输出为 $f_l(\mathrm{x}) = 1^\top \mathrm{x}^l$，且第 $_i$ 层

定义为 $\mathrm{x}^i = \mathrm{x} \odot W^{(i-1)}\,\mathrm{x}^{i-1} + \mathrm{x}^{i-1}$。然后，多元多项式 $f_l(\mathrm{x})$ 在以下类别中重现多项式：

$$\left\{ \sum_{\boldsymbol{\alpha}} c_{\boldsymbol{\alpha}} \left(W^{(1)}, \ldots, W^{(l)}\right) x_1^{\alpha_1} x_2^{\alpha_2} \ldots x_d^{\alpha_d} \;\middle|\; 0 \le |\boldsymbol{\alpha}| \le l+1, \boldsymbol{\alpha} \in \mathbb{N}^d \right\},$$

其中 $c_{\boldsymbol{\alpha}} = \sum_{\mathrm{j} \in C_p^{|\boldsymbol{\alpha}|-1}} \sum_{\mathrm{i} \in P_{\boldsymbol{\alpha}}} \prod_{k=1}^{|\boldsymbol{\alpha}|-1} w_{i_k i_{k+1}}^{(j_k)} \cdot w_{ij}^{(k)}$，$\alpha_i$ 是矩阵 $W^{(k)}$ 的第 (i,j) 个元素，并且 $P_{\boldsymbol{\alpha}}$ = 置换 $(\cup_i \{\underbrace{i, \ldots, i}_{\alpha_i}\}$ 次 $0\})$.

定理4.2（逐特征）。在与定理4.1相同的设定下，我们进一步假设输入$\mathrm{x} = [\mathrm{x}_1; \ldots; \mathrm{x}_k]$包含$k$个特征嵌入，并将每个$\mathrm{x}_i$视为一个单元。那么，一个$l$层交叉网络的输出$\mathrm{x}^l$会生成所有阶数不超过$l+1$的特征交互。具体而言，对于特征其在$I$中的（重复）索引，令$P_I = Permutations(I)$，则它们的$p$阶交互由下式给出：

$$\sum_{\mathrm{i} \in P_I} \sum_{\mathrm{j} \in C_p^{p-1}} \mathrm{x}_{i_1} \odot \left(W_{i_1,i_2}^{(j_1)} \mathrm{x}_{i_2} \odot \ldots \odot \left(W_{i_k,i_{k+1}}^{(j_k)} \mathrm{x}_{i_{l+1}} \right) \right)$$

从逐位和逐特征两个角度来看，交叉网络能够为一个l层的交叉网络创建所有阶数高达$l+1$的特征交互。与DCN-V相比，DCN-V2以更多参数刻画相同的多项式类，并且更具表达力。此外，DCN-V2中的特征交互更具表达力，并且可以从逐位和逐特征两个角度看待，而在DCN中它仅是逐位的[26, 46, 50]。

4.2 与相关工作的联系

我们研究了DCN-V2与其他SOTA特征交互学习方法之间的联系；我们仅关注每个模型的特征交互组件，而忽略DNN组件。为了比较的目的，我们假设特征嵌入的大小相同，均为e。

DCN。我们提出的模型在很大程度上受到了DCN [50]的启发。让我们采用DCN [50]的高效投影视角，即它隐式地生成所有成对交叉，然后将其投影到一个低维空间；DCN-V2类似，但具有不同的投影结构。

$$\mathrm{x}_{\text{DCN}}^\top = \mathrm{x}_{\text{pairs}} \begin{bmatrix} \mathrm{w} & 0 & \ldots & 0 \\ 0 & \mathrm{w} & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \mathrm{w} \end{bmatrix}, \mathrm{x}_{\text{DCN-V2}}^\top = \mathrm{x}_{\text{pairs}} \begin{bmatrix} \mathrm{w}_1 & 0 & \ldots & 0 \\ 0 & \mathrm{w}_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \mathrm{w}_d \end{bmatrix}$$

其中 $\mathrm{xpairs} = [x_i \tilde{x}_j]_{\forall i,j}$ 包含 x_0 与 $\tilde{\mathrm{x}}$ 之间所有 d^2 个成对交互；$\mathrm{w} \in \mathbb{R}^d$ 是 DCN-V 中的权重向量；$\mathrm{w}_i \in \mathbb{R}^d$ 是 DCN-V2 中权重矩阵的第 i 列（Eq.(1)）。

DLRM 和 DeepFM。两者本质上都是不含 DNN 组件的二阶 FM（忽略细微差异）。因此，我们简化分析，并与公式为 $\mathrm{x}^\top \boldsymbol{\beta} + \sum_{i<j} w_{ij} \langle \mathrm{x}_i, \mathrm{x}_j \rangle$ 的 FM 进行比较。这等价于具有结构化权重矩阵的单层 DCN-V2（式 (1) 去掉残差项）。

$$1^\top \left(\begin{bmatrix} \mathrm{x}_1 \\ \mathrm{x}_2 \\ \vdots \\ \mathrm{x}_k \end{bmatrix} \odot \left(\begin{bmatrix} 0 & w_{12} I & \cdots & w_{1k} I \\ 0 & 0 & \cdots & w_{2k} I \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & 0 \end{bmatrix} \begin{bmatrix} \mathrm{x}_1 \\ \mathrm{x}_2 \\ \vdots \\ \mathrm{x}_k \end{bmatrix} + \begin{bmatrix} \boldsymbol{\beta}_1 \\ \boldsymbol{\beta}_2 \\ \vdots \\ \boldsymbol{\beta}_k \end{bmatrix} \right) \right)$$

xDeepFM。第k层的第h个特征图由下式给出：

$$\mathrm{x}_{h,*}^k = \sum_{i=1}^{k-1} \sum_{j=1}^{m} w_{ij}^{k,h} (\mathrm{x}_{i,*}^{k-1} \odot \mathrm{x}_j)$$

第1层的第h个特征图等价于1层DCN-V2（无残差项的式(1)）。

$$\mathrm{x}_{h,*}^1 = [I, I, \cdots, I]\,(\mathrm{x} \odot (W\mathrm{x})) = \sum_{i=1}^{k} \mathrm{x}_i \odot (W_{i,:}\mathrm{x})$$

其中第 (i,j) 个块 $W_{i,j} = w_{ij} \cdot I$，以及 $W_{i,:} \coloneqq [W_{i,1}, \ldots, W_{i,k}]$。

AutoInt。AutoInt的交互层采用了多头自注意力机制。为简单起见，我们假设AutoInt中使用单头；多头情况可以通过拼接交叉层进行简要比较。

从高层视角来看，AutoInt的第一层输出$\tilde{}\mathrm{x} = [\tilde{}\mathrm{x}_1; \tilde{}\mathrm{x}_2; \ldots; \tilde{}\mathrm{x}_k]$，其中$\tilde{}\mathrm{x}_i$编码了所有与第i个特征相关的二阶特征交互。然后，$\tilde{}\mathrm{x}$被送入第二层以学习更高阶的交互。这与DCN-V2相同。

从低层次视角来看（忽略残差项），

$$\begin{aligned} \widetilde{\mathrm{x}}_i &= ReLU\left(\sum_{j=1}^{k} \frac{\exp\left(\langle W_{\mathrm{q}}\mathrm{x}_i, W_{\mathrm{k}}\mathrm{x}_j \rangle\right)}{\sum_j \exp\left(\langle W_{\mathrm{q}}\mathrm{x}_i, W_{\mathrm{k}}\mathrm{x}_j \rangle\right)} (W_{\mathrm{v}}\mathrm{x}_j) \right) \\ &= ReLU\left(\sum_{j=1}^{k} \mathrm{softmax}(\mathrm{x}_i^\top \widetilde{W} \mathrm{x}_j)\; W_{\mathrm{v}}\mathrm{x}_j \right) \end{aligned}$$

其中 $\langle \cdot, \cdot \rangle$ 表示内积（点积），以及 $\widetilde{W} = W_{\mathrm{q}} W_{\mathrm{k}}$。而在 DCN-V2 中，

$$\widetilde{\mathrm{x}}_i = \sum_{j=1}^{k} \mathrm{x}_i \odot (W_{i,j}\mathrm{x}_j) = \mathrm{x}_i \odot (W_{i,:}\mathrm{x}) \tag{5}$$

其中 $W_{i,j}$ 表示 W 的第 (i,j) 个块。显然，差异在于我们如何建模特征交互。AutoInt 声称非线性来自 ReLU(·)；我们认为每个求和项也有贡献。不同的是，DCN-V2 使用了 $\mathrm{x}_i \odot W_{i,j}\mathrm{x}_j$。

PNN。内积版本（IPNN）与FM类似。对于外积版本（OPNN），它首先显式地创建所有d^2成对交互，然后使用一个d'乘d^2的稠密矩阵将它们投影到一个更低维的空间d'。不同的是，DCN-V2使用结构化矩阵隐式地创建交互。

5 研究问题

我们有兴趣为以下研究问题寻找答案：

RQ1 特征交互学习方法何时会比基于ReLU的DNN更高效？RQ2 每个基线的特征交互组件在不与DNN集成时表现如何？RQ3 所提出的mDCN方法与基线相比如何？我们能否通过mDCN和低秩DCN的混合在模型准确性和成本之间实现更健康的权衡？RQ4 mDCN中的设置如何影响模型质量？RQ5 mDCN是否捕捉到了重要的特征交叉？模型是否提供了良好的可理解性?

在整篇论文中，“CrossNet”或“CN”表示交叉网络；后缀“Mix”表示低秩版本的混合。

6 特征交叉技术的实证理解（RQ1）

许多近期工作[1, 6, 13, 26, 34, 35, 50]提出对显式特征交叉进行建模，而这些特征交叉无法从传统神经网络中高效学习。然而，大多数工作仅研究了具有未知交叉模式和噪声数据的公共数据集；很少有工作在具有已知真实模型的干净环境中进行研究。因此，理解以下内容很重要：1）在哪些情况下传统神经网络会变得低效；2）DCN-V2的交叉网络中每个组件的作用。

我们在DCN模型中使用交叉网络来表示那些特征交叉方法，并与ReLU进行比较，ReLU在工业推荐系统中被广泛使用。为了简化实验并便于理解，我们假设每个特征x_i的维度为一，单项式$x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_d^{\alpha_d}$表示特征之间的$|\boldsymbol{\alpha}|$阶交互。

随难度增加的性能。仅考虑二阶特征交叉，并令真实模型为 $f(\mathrm{x}) = \sum_{|\boldsymbol{\alpha}|=2} w_{\boldsymbol{\alpha}} x_1^{\alpha_1} x_2^{\alpha_2} \ldots x_d^{\alpha_d}$。那么，学习 $f(\mathrm{x})$ 的难度取决于：1）稀疏性（$w_{\boldsymbol{\alpha}} = 0$）、交叉的数量，以及2）交叉模式的相似性（由 $\mathrm{Var}(w_{\boldsymbol{\alpha}})$ 表征），这意味着一个特征的变化会以相似幅度同时影响大多数特征交叉。我们在式（6）中创建难度递增的合成数据集。

$$
\begin{aligned}
f_1(\mathrm{x}) &= x_1^2 + x_1 x_2 + x_3 x_1 + x_4 x_1 \\
f_2(\mathrm{x}) &= x_1^2 + 0.1 x_1 x_2 + x_2 x_3 + 0.1 x_3^2 \\
f_3(\mathrm{x}) &= \sum_{(i,j)\in S} w_{ij} x_i x_j, \quad \mathrm{x} \in \mathbb{R}^{100}, |S| = 100
\end{aligned}
\tag{6}
$$

其中集合 S 和权重 w_{ij} 是随机分配的，而 x_i 则从区间 [-1, 1] 中均匀采样。

表1报告了5次运行的平均RMSE和模型大小。当交叉模式简单（f_1）时，DCN-V2和DCN都高效。当模式变得更复杂（f_3）时，DCN-V2保持准确，而DCN性能下降。即使采用更宽更深的结构（f_1和f_2的层大小为[200, 200]，f_3为[1024, 512, 256]），DNN的性能仍然较差。这表明DNN在建模单项式模式方面效率低下。

表1：难度递增的多项式拟合的RMSE和模型大小（#参数）。

	DCN (1Layer)		DCN-V2 (1Layer)		DNN (1Layer)		DNN (large)	
	RMSE	Size	RMSE	Size	RMSE	Size	RMSE	Size
f_1	8.9E-13	12	**5.1E-13**	24	2.7E-2	24	4.7E-3	41K
f_2	1.0E-01	9	**4.5E-15**	15	3.0E-2	15	1.4E-3	41K
f_3	2.6E+00	300	**6.7E-07**	10K	2.7E-1	10K	7.8E-2	758K

每个组件的作用。我们还分别对3阶和4阶齐次多项式进行了消融研究。对于每个阶数，我们从$\mathrm{x} \in \mathbb{R}^{50}$中随机选择了20个交叉项。

图4展示了平均RMSE随层深的变化。显然，$\mathrm{x}_0 \odot (W \mathrm{x}_i)$ 模型在层d-1处出现d阶交叉，这一点通过3阶多项式在第2层达到最佳性能得到验证（4阶类似）。然而，在其他层，性能显著下降。这正是偏置项和残差项发挥作用的地方——它们创建并维持直到最高阶的所有交叉。这缩小了层，并在引入冗余交叉时稳定模型。这对于具有未知交叉模式的现实世界应用尤为重要。

图4还揭示了DCN在建模复杂交叉模式时表达能力有限。

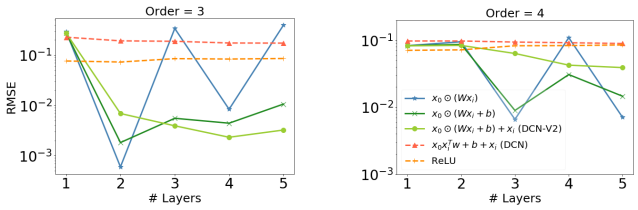


图4：3阶和4阶齐次多项式拟合。x轴表示使用的层数；y轴表示RMSE（越低越好）。在图例中，前3个模型是包含不同组件的DCN-V2。

随着层深度增加的性能。我们现在研究更接近真实世界的场景，其中交叉项具有组合阶数。

$$
f(\mathrm{x}) = \mathrm{x}^\top \mathrm{w} + \sum_{\boldsymbol{\alpha}\in S} w_{\boldsymbol{\alpha}} x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_d^{\alpha_d} + 0.1 \sin(2\mathrm{x}^\top \mathrm{w_s} + 0.1) + 0.01\epsilon
$$

其中随机选择的集合 $S = S_2 \cup S_3 \cup S_4$、$|S_2| = 20, |S_3| = 10$, $|S_4| = 5$ 和 $\forall \boldsymbol{\alpha} \in S_i, |\boldsymbol{\alpha}| = i$；正弦引入扰动，$\epsilon$ 表示高斯噪声。

表2报告了5次运行的平均RMSE。随着层深度的增加，CN-M能够捕获数据中更高阶的特征交叉，从而提升了性能。得益于偏置项和残差项，性能在第3层之后并未下降，尽管此时引入了冗余的特征交互。

表2：组合阶（1 - 4）多项式拟合。

#Layers	1	2	3	4	5
DCN-V2	1.43E-01	2.89E-02	**9.82E-03**	9.87E-03	9.92E-03
DNN	1.32E-01	1.03E-01	1.03E-01	1.09E-01	1.05E-01

总之，即使使用更深更大的网络，ReLU在捕捉显式特征交叉（乘法关系）方面效率低下。这与先前的研究[1]高度一致。当交叉模式变得更加复杂时，准确率显著下降。DCN能准确捕捉简单的交叉模式，但在更复杂的模式上失败。另一方面，DCN-V2对于复杂的交叉模式仍然保持准确和高效。

7 实验结果（RQ2 - RQ5）

本节通过3个数据集和2个平台上的实验，实证验证了DCN-V2在特征交互学习方面相较于SOTA的有效性。鉴于近期对已发表结果可复现性不佳的担忧[8, 33, 38]，我们进行了一项公平且全面的实验研究，通过广泛的超参数搜索来适当调整所有基线方法和所提出的方法。此外，对于每个最优设置，我们使用不同的随机初始化训练5个模型，并报告均值和标准差。

第7.2节研究了特征交叉学习组件的性能（RQ2），比较了未与DNN ReLU层集成的基线（类似于[26, 46]）；为了进行清晰的比较，仅考虑了稀疏特征。第7.3节全面比较了DCN-V2与所有基线（RQ3）。第7.5节评估了超参数对DCN-V2性能的影响（RQ4）。第7.6节侧重于模型理解（RQ5），即我们是否确实通过DCN-V2发现了有意义的特征交叉。

7.1 实验设置

本节描述了实验设置，包括训练数据集、基线方法，以及超参数搜索和训练过程的细节。

7.1.1 数据集。表3列出了每个数据集的统计信息：

表3：数据集。

Data	# Examples	# Features	Vocab Size
Criteo	45M	39	2.3M
MovieLen-1M	740k	7	3.5k
Production	> 100B	NA	NA

Criteo3。最流行的点击率（CTR）预测基准数据集包含7天内的用户日志。我们遵循[46, 50]，使用前6天进行训练，并将最后一天的数据随机等分为验证集和测试集。我们对13个连续特征进行对数归一化（特征2使用$\log(x+4)$，其他特征使用$\log(x+1)$），并嵌入剩余的26个类别特征。

MovieLen-1M4。推荐系统研究中最流行的数据集。每个训练样本包含一个⟨用户特征、电影特征、评分⟩三元组。与AutoInt [46]类似，我们将该任务形式化为一个回归问题。所有评分为1和2的均归一化为0；评分为4和5的归一化为1；评分为3的则被移除。使用了6个非多值类别特征并对其进行嵌入。数据被随机划分为80%用于训练、10%用于验证和10%用于测试。

7.1.2 基线。我们将提出的方法与6种SOTA特征交互学习算法进行比较。表4简要概述了这些方法之间的比较。

7.1.3 实现细节。所有基线方法和我们的方法均在TensorFlow v1中实现。为了公平比较，除特征交互组件5外，所有模型的所有实现均保持一致。

嵌入。除DNN和DCN外，所有基线方法都要求每个特征的嵌入大小相同。因此，我们将所有模型的嵌入大小固定为Criteo的Avg$\left(\sum \text{vocab } 6 \cdot (\text{vocab cardinality})^{\frac{1}{4}}\right)$ (39和Movielen-1M)的30。

[3]http://labs.criteo.com/2014/02/kaggle-display-advertising-challenge-dataset
[4]https://grouplens.org/datasets/movielens
[5]We adopted implementation from https://github.com/Leavingseason/xDeepFM, https://github.com/facebookresearch/dlrm and https://github.com/shenweichen/DeepCTR
[6]This formula is a rule-of-thumb number that is widely used [50], also see https://developers.googleblog.com/2017/11/introducing-tensorflow-feature-columns.html

表4：模型之间的高层比较。假设输入 $\mathrm{x}_0 = [\mathrm{v}_1; \ldots; \mathrm{v}_k]$ 包含 k 个特征嵌入，每个特征嵌入表示为 v_i。$\oplus$ 表示拼接；$\otimes$ 表示外积；$\odot$ 表示哈达玛积。$f_i(\cdot)$ 表示隐式特征交互，即 ReLU 层。在最后一列中，“+”符号位于 logit 层级。

Model	Explicit Interactions (f_e)		Final Objective
	Order	(Simplified) Key Formula	
PNN [35]	2	$\mathrm{x}_o = [\mathrm{v}_i^\top \mathrm{v}_j \mid \forall i,j]$ (IPNN) $\mathrm{x}_o = [\mathrm{vec}(\mathrm{v}_i \otimes \mathrm{v}_j) \mid \forall i,j]$ (OPNN)	$f_i \circ f_e$
DeepFM [13]	2	$\mathrm{x}_o = [\mathrm{v}_i^\top \mathrm{v}_j \mid \forall i,j]$	$f_i + f_e$
DLRM [34]	2	$\mathrm{x}_o = [\mathrm{v}_i^\top \mathrm{v}_j \mid \forall i,j]$	$f_i \circ f_e$
DCN [50]	≥ 2	$\mathrm{x}_{i+1} = \mathrm{x}_0 \otimes \mathrm{x}_i \mathrm{w}_i$	$f_i + f_e$
xDeepFM [26]	≥ 2	$\mathrm{v}_h^k = \sum_{i,j} \boldsymbol{w}_{ij}^{kh} (\mathrm{v}_i^{k-1} \odot \mathrm{v}_j)$	$f_i + f_e$
AutoInt [46]	NA	$\widetilde{\mathrm{v}}_i = g\left(\frac{\sum_j \exp(\langle W_q \mathrm{v}_i, W_k \mathrm{v}_j\rangle) W_o \mathrm{v}_j}{\sum_j \exp(\langle W_q \mathrm{v}_i, W_k \mathrm{v}_j\rangle)}\right)$	$f_i + f_e$
DCN-V2 (ours)	≥ 2	$\mathrm{x}_i = \mathrm{x}_0 \odot (W_i \mathrm{x}_i)$	$f_i \circ f_e$ $f_i + f_e$

优化。我们使用了Adam [22]，批量大小为512（MovieLen为128）。卷积核采用He Normal [15]初始化，偏置初始化为0；梯度裁剪范数为10；对训练参数应用了衰减率为0.9999的指数移动平均。

可复现性与公平比较：超参数调优与结果报告。对于所有基线方法，我们首先在较大范围内对超参数进行粗粒度网格搜索，随后在较小范围内进行细粒度搜索。为确保可复现性并降低模型方差，对于每种方法和每个数据集，我们报告最佳配置在5次独立运行中的均值和标准差。下面我们详细描述Criteo的设置；对于MovieLens，我们采用类似流程但使用不同的范围。

我们首先描述跨基线共享的超参数。学习率在对数尺度上从10^{-4}调整到10^{-1}，然后在线性尺度上缩小到10^{-4}至5×10^{-4}。训练步数在{150k, 160k, 200k, 250k, 300k}范围内搜索。隐藏层数在{1, 2, 3, 4}范围内，其层大小在{562, 768, 1024}范围内。正则化参数λ在$\{0, 3 \times 10^{-5}, 10^{-4}\}$范围内。

然后我们描述每个模型自身的超参数，其中搜索空间是根据已报告的设置设计的。对于DCN，交叉层数范围为1到4。对于AutoInt，注意力层数为2到4；注意力嵌入大小在{20, 32, 40}中；注意力头数为2到3；残差连接为开或关。对于xDeepFM，CIN层大小在{100, 200}中，深度在{2, 3, 4}中，激活函数为恒等，计算方式为直接或间接。对于DLRM，底层MLP层大小和数量在{(512,256,64), (256,64)}中。对于PNN，我们运行了IPNN、OPNN和PNN*，对于后两者，核类型在{全矩阵, 向量, 数量}中。对于所有模型，总参数数量上限为$1024^2 \times 5$，以限制搜索空间并避免过于昂贵的计算。

7.2 单独特征交互组件的性能（RQ2）

我们仅考虑每个模型的特征交互组件，而不考虑其DNN组件。此外，我们仅考虑类别特征，因为密集特征在各基线中的处理方式不同。表5展示了在Criteo数据集上的结果。每个基线的调参方式与第7.1.3节类似。主要有两个观察结果。1）高阶方法表现出优于二阶方法的性能。这表明高阶交叉在该数据集中是有意义的。2）在高阶方法中，交叉网络取得了最佳性能，与DNN相比持平或略优。

表5：各模型特征交互组件的LogLoss（测试）（无DNN）。仅使用类别特征。在“Setting”列中，l表示层数。

	Model	LogLoss	Best Setting
2nd	PNN [35]	0.4715 ± 4.430e-04	OPNN, kernel=matrix
	FM	0.4736 ± 3.04E-04	–
>2	CIN [26]	0.4719 ± 9.41E-04	l=3, cinLayerSize=100
	AutoInt [46]	0.4711 ± 1.62E-04	l=2, head=3, attEmbed=40
	DNN	0.4704 ± 1.57E-04	l=2, size=1024
	CrossNet	0.4702 ± 3.80E-04	l=2
	CrossNet-Mix	**0.4694 ± 4.35E-04**	l=5, expert=4, gate=$\frac{1}{1+e^{-x}}$

7.3 基线性能（RQ3）

本节以端到端的方式比较了DCN-V2方法与基线之间的性能。请注意，每个模型报告的最佳设置是在包括基线在内的广泛模型容量和超参数空间中搜索得到的。如果两种设置表现相当，我们报告成本较低的设置。表6展示了Criteo和MovieLen测试集上的最佳LogLoss和AUC（ROC曲线下面积）。对于Criteo，0.001级别的提升被认为是显著的（见[13, 46, 50]）。我们可以看到，DCN-V2始终优于基线（包括DNN），并实现了良好的质量/成本权衡。还值得一提的是，表6中报告的基线性能相比之前论文报告的数字有所提升（见附录中的表9）；然而，当与DNN集成时，它们的性能差距正在缩小（与表5相比），其表现与基于ReLU的DNN相当，有时甚至更差，即使进行了细粒度的模型调优。

最佳设置。最优超参数见表6。对于DCN-V2模型，“堆叠”和“并行”结构均优于所有基线，而“堆叠”在Criteo上表现更好，“并行”在Movielen-1M上表现更好。在Criteo上，设置为：DCN-Mix的门控为常数、激活函数为hard_tanh；CrossNet的门控为softmax、激活函数为identity。所有基线的最佳训练步数均为150k；所有模型的学习率各不相同。

模型质量——基线之间的比较。当将特征交叉学习组件与DNN集成时，高阶方法的优势不那么明显，并且所有模型之间的性能差距在Criteo上正在缩小（相比（见表5）。这表明了隐式特征交互的重要性以及良好调优的DNN模型的强大能力。

对于二阶方法，DLRM的表现不如DeepFM，尽管它们都源自FM。这可能是因为DLRM在点积层之后省略了一阶稀疏特征。PNN模型对二阶交叉的表达能力更强，在MovieLen-1M上表现更好；然而在Criteo上，其平均LogLoss因高标准差而被推高。对于高阶方法，xDeepFM、AutoInt和DCN在Criteo上表现相似，而在MovieLens上xDeepFm显示出高方差。

DCN-V2通过显式建模高达三阶的交叉特征（超越了DNN所隐含的交叉），取得了最佳性能（在Criteo数据集上，0.001被视为显著[26, 46, 50]）。DCN-Mix作为低秩DCN的混合，有效利用了内存，并在保持准确率的同时将成本降低了30%。有趣的是，仅CrossNet在两个数据集上均优于DNN；更多讨论见第7.4节。

模型质量——与DNN的比较。DNN是通用逼近器，在高度优化时是难以击败的基线。因此，我们对DNN及所有基线进行了精细调优，并使用了比文献中（例如[26, 46]中的200 - 400）更大的层规模。令我们惊讶的是，DNN与大多数基线表现不相上下，甚至超越了某些模型。

我们的假设是，基线中那些显式的特征交叉并未以表达力强且易于优化的方式建模。前者使得其性能很容易被大容量DNN所匹配。后者则容易导致可训练性问题，使模型不稳定，难以找到良好的局部最优或难以泛化。因此，当与DNN集成时，整体性能由DNN组件主导。在大容量DNN的情况下尤其如此，因为大容量DNN已经能够近似一些简单的交叉模式。

在表达能力方面，考虑二阶方法。PNN模型比DeepFM和DLRM更能表达交叉特征，这使其在MovieLen-1M上表现出色。这也解释了DCN相较于DCN-V2性能较差的原因。

就可训练性而言，某些模型可能本质上更难训练，并导致性能不尽如人意。以PNN为例。在MovieLen-1M上，它优于DNN，表明这些二阶交叉的有效性。然而，在Criteo上，PNN的优势已经减弱，平均性能与DNN相当。这是由PNN的不稳定性造成的。尽管其最佳运行结果优于DNN，但其多次试验的高标准差推高了平均损失。xDeepFM也存在可训练性问题（参见其在MovieLens上的高标准差）。在xDeepFM中，每个特征图编码了所有成对交叉，但仅依赖单个变量来学习每个交叉的重要性。在实践中，当与数量级更多的参数联合训练时，单个变量很难被学习。然后，一个学习不当的变量会导致噪声。

另一方面，DCN-V2始终优于DNN。它成功利用了显式和隐式特征交互。我们将此归因于交叉网络和深度网络之间平衡的参数数量（表达能力强），以及交叉网络的简单结构便于优化（易于优化）。值得注意的是，高层

DCN-V2的结构与AutoInt中采用的自注意力机制有着相似的精神，其中每个特征嵌入都关注其他特征的加权组合。不同之处在于，在注意力过程中，DCN-V2显式地建模了高阶交互，而AutoInt则是隐式地建模。

模型效率。表6还提供了模型大小和FLOPS7的详细信息。所报告的设置已针对每个模型和DNN组件的超参数进行了适当调优。对于大多数模型，FLOPS大致是#params的2倍；然而，对于xDeepFM，FLOPS高出一个数量级，使其在工业规模应用中不切实际（在[46]中也有观察到）。请注意，对于DeepFM和DLRM，我们也搜索了更大容量的模型；然而，它们并未带来更好的质量。在所有方法中，DCN-V2在保持相对高效的同时提供了最佳性能；DCN-Mix进一步降低了成本，在模型效率和质量之间实现了更好的权衡。

7.4 跨层能否替代ReLU层?

DCN-V2方法的稳定性能启发了我们进一步研究其交叉层（CrossNet）在学习显式高阶特征交叉方面的效率。

在资源受限的现实环境中，我们往往需要限制模型容量。因此，我们将模型容量（内存/参数数量）固定在不同水平，并比较了仅包含交叉层的模型（Cross Net）与基于ReLU的DNN之间的性能。表7报告了不同内存约束下的最佳测试LogLoss。内存通过改变交叉层数量及其秩（{128, 256}）、隐藏层数量及其大小来控制。最佳性能由交叉网络（5层）实现，这表明真实模型可以被多项式很好地近似。此外，每个内存限制下的最佳性能也由交叉网络实现，表明其既具有扎实的有效性又具有高效性。

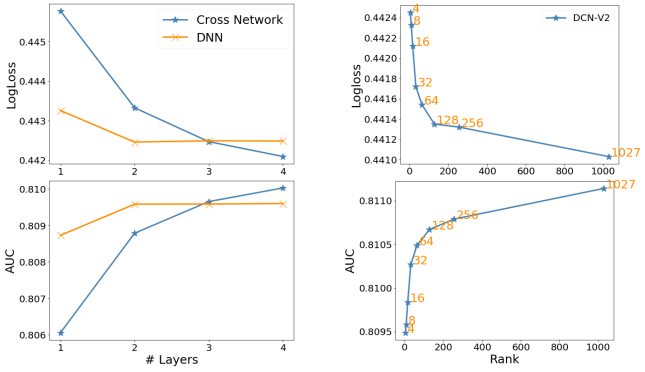
众所周知，ReLU层是包括DNN、循环神经网络（RNN）[18, 32, 40]和卷积神经网络（CNN）[23, 24, 42]在内的各种神经网络模型的主干。对我们来说，在某些应用中，我们可能完全用交叉层替代ReLU层，这既令人惊讶又令人鼓舞。显然，我们需要进行更多的分析和实验来验证这一假设。尽管如此，这是一项非常有趣的初步研究，为我们未来对交叉层的探索提供了启示。

7.5 超参数选择如何影响DCN-V2模型性能（RQ4）

本节考察模型性能随超参数的变化，这些超参数包括：1）交叉层的深度；2）DCN-Mix 的矩阵秩；3）DCN-Mix 中专家数量。

交叉层深度。按照设计，交叉网络捕获的最高特征交叉阶数随层深度增加而增加。因此，我们将自己限制在全秩交叉层，并评估性能随层深度的变化。

图5a展示了在Criteo数据集上增加层深度时的测试LogLoss和AUC。我们看到更深的交叉网络带来了稳定的质量提升，表明它能够捕获更多有意义的交叉项。然而，当使用更多层时，改进的速度减慢了。这表明高阶交叉项的贡献不如低阶交叉项显著。我们还使用了一个相同规模的DNN作为参考。当有≤ 2层时，DNN优于交叉网络；当有更多层可用时，交叉网络开始缩小性能差距，甚至超过了DNN。在小层数情况下，交叉网络只能近似非常低阶的交叉项（例如，1 ~ 2）；在大层数情况下，那些低阶交叉项被更多的参数所表征，并且那些高阶交互开始被捕捉。



(a) Layer depth **(b) Matrix rank**

图5：对数损失与AUC（测试）随深度和矩阵秩的变化。

矩阵的秩。权重矩阵的秩控制了参数数量以及通过交叉层的低频信号部分。因此，我们研究其对模型质量的影响。该模型基于一个表现良好的设置，包含3个交叉层，随后是3个大小为512的隐藏层。我们将每个交叉层中的稠密矩阵W近似为$UV^\top$，其中$U, V \in \mathbb{R}^{d\times r}$，并且我们变化$r$。我们宽松地将较小的维度$r$视为秩。

图5b展示了在Criteo数据集上测试LogLoss和AUC随矩阵秩r的变化情况。当r小至4时，其性能与其他基线相当。当r从4增加到64时，LogLoss几乎随r (线性下降，即模型性能提升)。当r从64进一步增加到满秩时，LogLoss的改善放缓。我们将64称为阈值秩。从64开始显著放缓表明，表征特征交叉的重要信号可以在前64个奇异值中被捕获。

我们对该阈值秩的取值的假设是 $O(k)$，其中 k 表示特征数（Criteo 为 39）。考虑矩阵 W 的第 (i, j) 个块，我们可以将 $W_{i,j} = W^L_{i,j} + W^H_{i,j}$ 视为，其中 $W^L_{i,j}$ 存储主导信号（低频），$W^H_{i,j}$ 存储其余部分（高频）。在最简单的情况下，$W^L_{i,j} = c_{ij}\,1\,1^\top$，整个矩阵 W^L 的秩将为 k。该假设的有效性仍有待跨多个数据集进行验证。

专家数量。我们研究了低秩专家数量对质量的影响。我们观察到：1）表现最佳的设置（#专家、门控、矩阵激活类型）因数据集和模型架构而异；2）表现最佳的模型

[7] FLOPS is a close estimation of run time, which is subjective to implementation details.

表6：在Criteo和Movielen-1M上的LogLoss和AUC（测试）。指标是在5次独立运行中取平均值，括号内为标准差。在“最佳设置”列中，左侧报告DNN设置，右侧报告模型特定设置。l表示层深度；n表示CIN层大小；h和e分别表示头数和注意力嵌入大小；K表示专家数，r表示总秩。

Baseline	Criteo						MovieLens-1M			
	Logloss	AUC	Params	FLOPS	Best Setting		Logloss	AUC	Params	FLOPS
PNN	0.4421 (5.8E-4)	0.8099 (6.1E-4)	3.1M	6.1M	(3, 1024)	OPNN	0.3182 (1.4E-3)	0.8955 (3.3E-4)	54K	110K
DeepFm	0.4420 (1.4E-4)	0.8099 (1.5E-4)	1.4M	2.8M	(2, 768)	–	0.3202 (1.0E-3)	0.8932 (7.7E-4)	46K	93K
DLRM	0.4427 (3.1E-4)	0.8092 (3.1E-4)	1.1M	2.2M	(2, 768)	[512,256,64]	0.3245 (1.1E-3)	0.8890 (1.1E-3)	7.7K	16K
xDeepFm	0.4421 (1.6E-4)	0.8099 (1.8E-4)	3.7M	32M	(3, 1024)	l=2, n=100	0.3251 (4.3E-3)	0.8923 (8.6E-4)	160K	990K
AutoInt+	0.4420 (5.7E-5)	0.8101 (2.6E-5)	4.2M	8.7M	(4, 1024)	l=2, h=2, e=40	0.3204 (4.4E-4)	0.8928 (3.9E-4)	260K	500K
DCN	0.4420 (1.6E-4)	0.8099 (1.7E-4)	2.1M	4.2M	(2, 1024)	l=4	0.3197 (1.9E-4)	0.8935 (2.1E-4)	110K	220K
DNN	0.4421 (6.5E-5)	0.8098 (5.9E-5)	3.2M	6.3M	(3, 1024)	–	0.3201 (4.1E-4)	0.8929 (2.3E-4)	46K	92K
Ours										
DCN-V2	**0.4406 (6.2E-5)**	**0.8115 (7.1E-5)**	3.5M	7.0M	(2, 768)	l=2	0.3170 (3.6E-4)	0.8950 (2.7E-4)	110K	220K
DCN-Mix	0.4408 (1.0E-4)	0.8112 (9.8E-5)	2.4M	4.8M	(2, 512)	l=3, K=4, r=258	**0.3160 (4.9E-4)**	**0.8964 (2.9E-4)**	110K	210K
CrossNet	0.4413 (2.5E-4)	0.8107 (2.4E-4)	2.1M	4.2M	–	l=4, K=4, r=258	0.3185 (3.0E-4)	0.8937 (2.7E-4)	65K	130K

表7：固定内存预算下的Logloss和AUC（测试）。

#Params		7.9E+05	1.3E+06	2.1E+06	2.6E+06
LogLoss	CrossNet	**0.4424**	**0.4417**	**0.4416**	**0.4415**
	DNN	0.4427	0.4426	0.4423	0.4423
AUC	CrossNet	**0.8096**	**0.8104**	**0.8105**	**0.8106**
	DNN	0.8091	0.8094	0.8096	0.80961

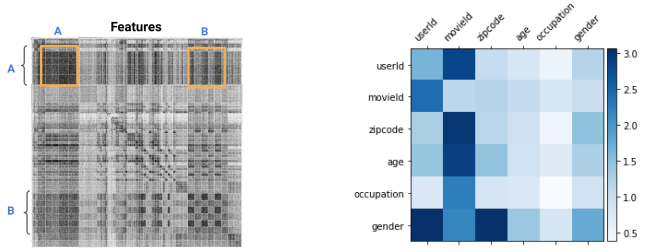
每种设置都产生了相似的结果。例如，对于在Criteo上总秩为256的2层交叉网络，1、4、8、16和32个专家的LogLoss分别为0.4418、0.4416、0.4416、0.4422和0.4420。更多低秩专家并未比单个更高秩专家表现更好，这一事实可能是由所采用的朴素门控函数和优化方法导致的。我们相信，更复杂的门控[21, 28, 29]和优化技术（例如替代训练、特殊初始化、温度调整）将能更多地利用专家混合模型。然而，这超出了本文的范围，我们将其留待未来工作。

7.6 模型理解（RQ5）

一个关键的研究问题是，所提出的方法是否确实在学习有意义的特征交叉。对学习到的特征交叉有良好的理解有助于提高模型的可解释性，并且对机器学习公平性和机器学习健康等领域尤为关键。幸运的是，DCN-V2中的权重矩阵W恰好揭示了模型已经学习到哪些特征交叉是重要的。具体而言，我们假设每个输入$\mathrm{x} = [\mathrm{x}_1; \mathrm{x}_2; \ldots; \mathrm{x}_k]$包含$k$个特征，每个特征由一个嵌入$\mathrm{x}_i$表示。那么，式(7)中特征交叉组件的分块视图（忽略偏置）表明，第i个和第j个特征之间特征交互的重要性由第(i, j)个块$W_{i,j}$来刻画。

$$\mathrm{x} \odot W\mathrm{x} = \begin{bmatrix} \mathrm{x}_1 \\ \mathrm{x}_2 \\ \vdots \\ \mathrm{x}_k \end{bmatrix} \odot \begin{bmatrix} W_{1,1} & W_{1,2} & \cdots & W_{1,k} \\ W_{2,1} & W_{2,2} & \cdots & W_{2,k} \\ \vdots & \vdots & \ddots & \vdots \\ W_{k,1} & W_{k,2} & \cdots & W_{k,k} \end{bmatrix} \begin{bmatrix} \mathrm{x}_1 \\ \mathrm{x}_2 \\ \vdots \\ \mathrm{x}_k \end{bmatrix} \tag{7}$$

图6展示了第一个交叉层中学习到的权重矩阵W。子图(a)展示了整个矩阵，橙色框突出显示了一些显著的特征交叉。非对角块对应于已知重要的交叉，表明DCN-V2的有效性。对角块表示自交互（x^2的）。子图(b)展示了每个块的Frobenius范数，并指示了一些学习到的强交互，例如，性别×用户ID，电影ID×用户ID。



(a) Production data **(b) Movielen-1M**

图6：DCN-V2中学习到的权重矩阵可视化。行和列表示真实特征。对于(a)，出于专有原因未显示特征名称；较暗的像素表示其绝对值中较大的权重。对于(b)，每个块表示每个矩阵块的Frobenius范数。

8 在谷歌实现 DCN-V2 的生产化

本节提供了一个案例研究，分享我们在Google大规模推荐系统中将DCN-V2生产化的经验。我们通过DCN-V2在离线模型准确性和在线关键业务指标上都取得了显著收益。

排序问题：给定一个用户和大量候选项目，我们的问题是返回用户最有可能与之互动的排名前k个项目。令训练数据表示为$\{(\mathrm{x}_i, y_i)\}_{i=1}^N$，其中$\mathrm{x}_i$表示多种模态的特征，例如用户的兴趣、项目的元数据和上下文特征；y_i是表示用户行为（例如点击）的标签。目标是学习

一个函数 $f: \mathbb{R}^d \mapsto \mathbb{R}$，用于预测概率 $P(y|x)$，即在给定特征 x 的情况下用户的行为 y。

生产数据与模型：生产数据是采样的用户日志，由数千亿个训练样本组成。稀疏特征的词汇量从2到数百万不等。基线模型是一个具有ReLU激活的全连接多层感知机（MLP）。

与生产模型的比较：与生产模型相比，DCN-V2在AUCLoss（1 - AUC）上提升了0.6%。对于这个特定模型，AUCLoss上0.1%的提升被认为是显著的改进。我们还观察到关键指标上的显著在线性能提升。表8通过用相同大小的ReLU层替换交叉层，进一步验证了DCN-V2带来的增益量。

表8：DCN-V2相对于同等规模ReLU的AUCLoss相对值

1layer ReLU	2layer ReLU	1layer DCN-V2	2layer DCN-V2
0%	-0.15%	-0.19%	-0.45%

实践经验。我们分享一些在生产环境中部署DCN-V2过程中学到的实践经验。

- 最好将交叉层插入到DNN的输入层与隐藏层之间（在[44]中亦有观察）。我们的假设是，随着离输入层越来越远，特征表示及其交互的物理意义变得越来越弱。
- 我们发现，通过堆叠或拼接 1 到 2 个交叉层，可以持续获得准确率提升。超过 2 个交叉层后，增益开始趋于平稳。
- 我们观察到，堆叠交叉层和拼接交叉层都效果良好。堆叠层学习高阶特征交互，而拼接层（类似于多头机制[48]）则捕捉互补的交互。
- 我们观察到，使用秩为（输入大小）/4的低秩DCN能够始终如一地保持全秩DCN-V2的准确性。

9 结论与未来工作

在本文中，我们提出了一个新模型——DCN-V2——以表达性强且简单的方式对显式交叉进行建模。观察到交叉网络中权重矩阵的低秩特性，我们还提出了一种低秩DCN的混合模型（DCN-Mix），以在模型性能和延迟之间实现更健康的权衡。DCN-V2已成功部署在多个网络规模的学习排序系统中，并取得了显著的离线模型准确率和在线业务指标提升。我们的实验结果也证明了DCN-V2相较于SOTA方法的有效性。

在未来工作中，我们有兴趣进一步加深对以下方面的理解：1）DCN-V2与优化算法（如二阶方法）之间的交互作用；2）嵌入、DCN-V2及其矩阵秩之间的关系。此外，我们希望改进DCN-Mix中的门控机制。而且，观察到DCN-V2中的交叉层可能作为DNN中ReLU层的潜在替代方案，我们非常有兴趣在更复杂的模型架构（例如RNN、CNN）中验证这一观察。

致谢。我们要感谢Bin Fu、Gang（Thomas）Fu和Mingliang Wang对DCN-V2的早期贡献；邓天硕、马文静、田亚阳、张淑英、张杰（Jerry）、Evan Ettinger、Samuel Ieong 以及许多其他人为 DCN-V2 的生产化所付出的努力和支持；陈婷提出了低秩混合的初步想法；以及唐嘉希提出的宝贵意见。

参考文献

[1] Alex Beutel, Paul Covington, Sagar Jain, Can Xu, Jia Li, Vince Gatto, and Ed H Chi. 2018. Latent cross: Making use of context in recurrent recommender systems. In Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining. 46–54. [2] Léon Bottou, Jonas Peters, Joaquin Quiñonero-Candela, Denis X Charles, D Max Chickering, Elon Portugaly, Dipankar Ray, Patrice Simard, and Ed Snelson. 2013. Counterfactual reasoning and learning systems: The example of computational advertising. The Journal of Machine Learning Research 14, 1 (2013), 3207–3260. [3] Andrei Z Broder. 2008. Computational advertising and recommender systems. In Proceedings of the 2008 ACM conference on Recommender systems. 1–2. [4] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In Proceedings of the 24th international conference on Machine learning. 129–136. [5] Ting Chen, Ji Lin, Tian Lin, Song Han, Chong Wang, and Denny Zhou. 2018. Adaptive mixture of low-rank factorizations for compact neural modeling. (2018). [6] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & Deep Learning for Recommender Systems. arXiv preprint arXiv:1606.07792 (2016). [7] Weiyu Cheng, Yanyan Shen, and Linpeng Huang. 2019. Adaptive Factoriza- tion Network: Learning Adaptive-Order Feature Interactions. arXiv preprint arXiv:1909.03276 (2019). [8] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are we really making much progress? A worrying analysis of recent neural recommen- dation approaches. In Proceedings of the 13th ACM Conference on Recommender Systems. 101–109. [9] Petros Drineas and Michael W Mahoney. 2005. On the Nyström method for approximating a Gram matrix for improved kernel-based learning. journal of machine learning research 6, Dec (2005), 2153–2175. [10] David Eigen, Marc’Aurelio Ranzato, and Ilya Sutskever. 2013. Learning factored representations in a deep mixture of experts. arXiv preprint arXiv:1312.4314 (2013). [11] Yuwei Fan, Jordi Feliu-Faba, Lin Lin, Lexing Ying, and Leonardo Zepeda-Núnez. 2019. A multiscale neural network based on hierarchical nested bases. Research in the Mathematical Sciences 6, 2 (2019), 21. [12] Gene H Golub and Charles F Van Loan. 1996. Matrix Computations Johns Hopkins University Press. Baltimore and London (1996). [13] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. arXiv preprint arXiv:1703.04247 (2017). [14] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. 2011. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. SIAM review 53, 2 (2011), 217–288. [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In Proceedings of the IEEE international conference on computer vision. 1026–1034. [16] Xiangnan He and Tat-Seng Chua. 2017. Neural factorization machines for sparse predictive analytics. In Proceedings of the 40th International ACM SIGIR conference on Research and Development in Information Retrieval. 355–364. [17] Jonathan L Herlocker, Joseph A Konstan, Loren G Terveen, and John T Riedl. 2004. Evaluating collaborative filtering recommender systems. ACM Transactions on Information Systems (TOIS) 22, 1 (2004), 5–53. [18] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. Neural computation 9, 8 (1997), 1735–1780. [19] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. Neural computation 3, 1 (1991), 79–87. [20] Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. 2014. Speeding up convo- lutional neural networks with low rank expansions. arXiv preprint arXiv:1405.3866 (2014). [21] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. arXiv preprint arXiv:1611.01144 (2016). [22] Diederik Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimiza- tion. arXiv preprint arXiv:1412.6980 (2014). [23] Steve Lawrence, C Lee Giles, Ah Chung Tsoi, and Andrew D Back. 1997. Face recognition: A convolutional neural-network approach. IEEE transactions on neural networks 8, 1 (1997), 98–113. [24] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. 1989. Backpropagation applied

到手写邮政编码识别。神经计算 1, 4 (1989), 541−551. [25] Zeyu Li, Wei Cheng, Yang Chen, Haifeng Chen, and Wei Wang. 2020. 通过层次注意力实现可解释的点击率预测。见第13届国际网络搜索与数据挖掘会议论文集。313−321. [26] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xdeepfm：结合显式和隐式特征交互的推荐系统。见第24届ACM SIGKDD国际知识发现与数据挖掘会议论文集。1754−1763. [27] Tie-Yan Liu. 2011. 信息检索中的排序学习。Springer Science & Business Media. [28] Christos Louizos, Max Welling, and Diederik P Kingma. 2017. 通过L_0正则化学习稀疏神经网络。arXiv预印本 arXiv:1712.01312 (2017). [29] Jiaqi Ma, Zhe Zhao, Jilin Chen, Ang Li, Lichan Hong, and Ed H Chi. 2019. Snr：用于多任务学习中灵活参数共享的子网络路由。见AAAI人工智能会议论文集，第33卷。216−223. [30] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. 使用多门专家混合模型建模多任务学习中的任务关系。见第24届ACM SIGKDD国际知识发现与数据挖掘会议论文集。1930−1939. [31] Hrushikesh N Mhaskar. 1996. 用于光滑解析函数最优逼近的神经网络。神经计算 8, 1 (1996), 164−177. [32] Tomá Mikolov, Stefan Kombrink, Luká Burget, Jan ernockỳ, and Sanjeev Khudanpur. 2011. 循环神经网络语言模型的扩展。见2011 IEEE国际声学、语音与信号处理会议（ICASSP）。IEEE, 5528−5531. [33] Kevin Musgrave, Serge Belongie, and Ser-Nam Lim. 2020. 度量学习现实检验。arXiv预印本 arXiv:2003.08505 (2020). [34] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. 用于个性化和推荐系统的深度学习推荐模型。arXiv预印本 arXiv:1906.00091 (2019). [35] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. 用于用户响应预测的基于产品的神经网络。见2016 IEEE第16届国际数据挖掘会议（ICDM）。IEEE, 1149−1154. [36] Steffen Rendle. 2010. 因子分解机。见2010 IEEE国际数据挖掘会议。IEEE, 995−1000. [37] Steffen Rendle. 2012. 使用libFM的因子分解机。ACM Trans. Intell. Syst. Technol. 3, 3, 文章57 (2012年5月), 22页. [38] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. 2020. 神经协同过滤与矩阵分解再审视。arXiv预印本 arXiv:2005.09683 (2020). [39] Paul Resnick and Hal R Varian. 1997. 推荐系统。Commun. ACM 40, 3 (1997), 56−58. [40] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1985. 通过误差传播学习内部表示。技术报告。加利福尼亚大学圣地亚哥分校拉霍亚认知科学研究所。 [41] J Ben Schafer, Joseph Konstan, and John Riedl. 1999. 电子商务中的推荐系统。见第1届ACM电子商务会议论文集。158−166. [42] Jürgen Schmidhuber. 2015. 神经网络中的深度学习：概述。神经网络 61 (2015), 85−117. [43] Frank Seide, Gang Li, Xie Chen, and Dong Yu. 2011. 用于会话语音转写的上下文相关深度神经网络中的特征工程。见2011 IEEE自动语音识别与理解研讨会。IEEE, 24−29. [44] Ying Shan, T Ryan Hoens, Jian Jiao, Haijing Wang, Dong Yu, and JC Mao. 2016. Deep Crossing：无需手工构造组合特征的网络规模建模。见第22届ACM SIGKDD国际知识发现与数据挖掘会议论文集。ACM, 255−262. [45] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. 极其庞大的神经网络：稀疏门控专家混合层。arXiv预印本 arXiv:1701.06538 (2017). [46] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. Autoint：通过自注意力神经网络进行自动特征交互学习。见第28届ACM国际信息与知识管理会议论文集。1161−1170. [47] Gregory Valiant. 2014. 使用神经网络学习多项式。(2014). [48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, ukasz Kaiser, and Illia Polosukhin. 2017. 注意力就是你所需要的一切。见神经信息处理系统进展。5998−6008. [49] Andreas Veit, Michael J Wilber, and Serge Belongie. 2016. 残差网络的行为类似于相对较浅网络的集成。见神经信息处理系统进展29，D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett (Eds.). Curran Associates, Inc., 550−558.

[50] Ruoxi Wang、Bin Fu、Gang Fu 和 Mingliang Wang。2017年。用于广告点击预测的深度与交叉网络。载于ADKDD’17会议论文集。1−7。[51] Ruoxi Wang、Yingzhou Li、Michael W Mahoney 和 Eric Darve。2019年。用于可扩展核评估的块基分解。SIAM J. Matrix Anal. Appl. 40, 4（2019），1497−1526。[52] Xiyu Yu、Tongliang Liu、Xinchao Wang 和 Dacheng Tao。2017年。关于通过低秩和稀疏分解压缩深度模型。载于IEEE计算机视觉与模式识别会议论文集。7370−7379。

附录

10 论文中报告的基线性能

表9列出了每项基线在论文中报告的Logloss和AUC指标。

11 定理证明

11.1 定理4.2的证明

证明。我们从记号开始；然后用归纳法证明。

符号说明。设 $[k] := \{1, \ldots, k\}$。将嵌入表示为 $\mathrm{x} = [\mathrm{x}_1; \mathrm{x}_2; \ldots; \mathrm{x}_c]$，第 l 个交叉层的输出为 $\mathrm{x}^l = [\mathrm{x}_1^l; \mathrm{x}_2^l; \ldots; \mathrm{x}_c^l]$，其中 $\mathrm{x}_i, \mathrm{x}_i^l \in \mathbb{R}^{e_i}$，且 e_i 是第 i 个特征的嵌入大小。为简化符号，我们还定义有序集合 I (（例如 (i_1, i_3, i_4))）中特征之间的特征交互，其权重由有序集合 J 表征，如下

$$g(I, J; \mathrm{x}, W) = \mathrm{x}_{i_1} \odot \left(W^{j_1}_{i_1, i_2} \mathrm{x}_{i_2} \odot \ldots \odot \left(W^{j_k}_{i_k, i_{k+1}} \mathrm{x}_{i_{l+1}} \right) \right) \quad (8)$$

其中权重 $W^j_{i_a, i_b}$ 表示第 j 个交叉层中权重 W^j 的第 (i_a, i_b) 个块，它有两个作用：对齐特征之间的维度，并增强特征交叉表示的印象。注意，给定 x_i 的顺序，矩阵 W 的下标是唯一确定的。

命题。我们首先通过归纳法证明 x_i^l 具有以下公式:

$$\mathrm{x}_i^l = \sum_{p=2}^{l+1} \sum_{I \in S_p^i} \sum_{J \in C_l^{p-1}} g(I, J; \mathrm{x}, W) + \mathrm{x}_i \quad (9)$$

其中 S_p^i 是一个集合，表示从 $[c]$ 中可重复地选取 p 个元素的所有组合，且第一个元素固定为 i: $S_p^i =: \{\mathrm{y} \in [c]^p \mid y_1 = i\}$, $\forall I \in S_p$, $I = (i_1, \ldots, i_p)$; 而 C_l^{p-1} 是一个集合，表示从整数 $[l]$ 中一次选取 $p-1$ 个索引的组合: $C_l^{p-1} := \{\mathrm{y} \in [l]^{p-1} \mid \forall i < j, y_i > y_j\}$. 基本情况。当 $l = 1$ 时，$\mathrm{x}_i^1 = \sum_j W_{i,j}^1 \mathrm{x}_j + \mathrm{x}_i$。
归纳步骤。假设当 $l = k$ 时，

$$\mathrm{x}_i^k = \sum_{p=2}^{k+1} \sum_{I \in S_p^i} \sum_{J \in C_k^{p-1}} g_J(\mathrm{x}; I) + \mathrm{x}_i$$

然后，对于 $l = k+1$，我们有

$$\begin{aligned}
\mathrm{x}_i^{k+1} &= \mathrm{x}_i \odot \sum_{q=1}^{c} W_{i,q}^{k+1} \mathrm{x}_q^k + \mathrm{x}_i^k \\
&= \mathrm{x}_i \odot \sum_{q=1}^{c} W_{i,q}^{k+1} \left(\sum_{p=2}^{k+1} \sum_{I \in S_p^q} \sum_{J \in C_k^{p-1}} g(I, J; \mathrm{x}, W) + \mathrm{x}_q \right) + \\
&\quad \sum_{p=2}^{k+1} \sum_{I \in S_p^i} \sum_{J \in C_k^{p-1}} g(I, J; \mathrm{x}, W) + \mathrm{x}_i \\
&= \sum_{q=1}^{c} \sum_{p=2}^{k+1} \sum_{I \in S_p^q} \sum_{J \in C_k^{p-1}} \mathrm{x}_i \odot \left(W_{i,q}^{k+1} g(I, J; \mathrm{x}, W) \right) + \\
&\quad \sum_{p=2} \sum_{J=k+1} \sum_{I \in S_2^i} g(I, J; \mathrm{x}, W) + \sum_{p=2}^{k+1} \sum_{I \in S_p^i} \sum_{J \in C_k^{p-1}} g(I, J; \mathrm{x}, W) + \mathrm{x}_i \\
&= \sum_{p=2}^{k+1} \sum_{J \in k+1 \oplus C_k^{p-1}} \sum_{I \in S_{p+1}^i} g(I, J; \mathrm{x}, W) + \\
&\quad \sum_{p=2} \sum_{J=k+1} \sum_{I \in S_2^i} g(I, J; \mathrm{x}, W) + \sum_{p=2}^{k+1} \sum_{I \in S_p^i} \sum_{J \in C_k^{p-1}} g(I, J; \mathrm{x}, W) + \mathrm{x}_i \\
&= \left(\sum_{p=3}^{k+2} \sum_{J \in k+1 \oplus C_k^{p-2}} \sum_{I \in S_p^i} + \sum_{p=3}^{k+1} \sum_{I \in S_p^i} \sum_{J \in C_k^{p-1}} \right) g(I, J; \mathrm{x}, W) + \\
&\quad \left(\sum_{p=2} \sum_{I \in S_2^i} \sum_{J \in C_k^1} g(I, J; \mathrm{x}, W) + \sum_{p=2} \sum_{J=k+1} \sum_{I \in S_2^i} \right) g(I, J; \mathrm{x}, W) + \mathrm{x}_i \\
&= \sum_{p=3}^{k+2} \sum_{J \in C_{k+1}^{p-1}} \sum_{I \in S_p^i} g(I, J; \mathrm{x}, W) + \sum_{p=2} \sum_{J = C_{k+1}^{p-1}} \sum_{I \in S_p^i} g(I, J; \mathrm{x}, W) + \mathrm{x}_i \\
&= \sum_{p=2}^{k+2} \sum_{I \in S_p^i} \sum_{J \in C_{k+1}^{p-1}} g(I, J; \mathrm{x}, W) + \mathrm{x}_i
\end{aligned}$$

其中 $\oplus$ 表示将索引 $k+1$ 添加到 C_k^{p-1} 集合中的每个元素。前 5 个等式是直接的。对于第 6 个等式，我们首先对第一项交换变量 $p' = p + 1$，并将第三项分为 $p = 2$ 和 $p > 2$ 的情况。然后，我们将这些项分为两种情况：$p = 2$ 和 $p > 2$。对于倒数第二个等式，我们合并了对 J 的求和。考虑从 $k+1$ 个整数中选择 $p-1$ 个索引的组合的集合，它可以分为两个集合，包含索引 $k+1$ 和不包含。因此，$C_{k+1}^{p-1} = C_k^{p-1} \cup \left((k+1) \oplus C_k^{p-2} \right)$。

结论。由于基础情况和归纳步骤均成立，我们得出 $\forall\, l \geq 1$，式 (9) 成立。至此证明完毕。在这种情况下，第 l 个交叉层包含所有阶数不超过 $l+1$ 的特征交互（按特征划分）。不同特征集之间的交互以不同方式参数化，具体而言，集合 I (feature 中的特征之间的交互可以重复)，阶数为 p 的交互为 $\sum_{i \in I'} \sum_{j \in I'} \{g(\mathrm{i}, \mathrm{j}; \mathrm{x}, W) = \mathrm{x}_{i_1} \odot \left(W^{j_1}_{i_1, i_2} \mathrm{x}_{i_2} \odot \ldots \odot \left(W^{j_k}_{i_k, i_{k+1}} \mathrm{x}_{i_{l+1}} \right) \right)\}$，其中 I' 包含 I 中元素的所有排列。

□

11.2 定理4.1的证明

证明。我们不再将每个特征嵌入视为一个单元，而是将输入嵌入 $\mathrm{x} = [x_1, x_2, \ldots, x_d]$ 中的每个元素 x_i 视为一个单元。这是定理4.2的一个特例，其中所有特征

表9：论文中报告的基线性能。指标（Logloss、AUC）引用自论文。每一行代表基线，每一列代表报告指标的论文。每个基线的最佳指标被标记用粗体表示。

Model \ Paper	DeepFM[13] (2017)	DCN[50] (2017)	xDeepFM[26] (2018)	DLRM[34] (2019)	AutoInt[46] (2019)	DCN-V2 (ours)
DeepFM	(0.45083, 0.8007)	–	(0.4468, 0.8025)	–	(0.4449, 0.8066)	**(0.4420, 0.8099)**
DCN	–	(0.4419, -)	(0.4467, 0.8026)	(-, ~ 0.789)	(0.4447, 0.8067)	**(0.4420, 0.8099)**
xDeepFM	–	–	(**0.4418**, 0.8052)	–	(0.4447, 0.8070)	(0.4421, **0.8099**)
DLRM	–	–	–	(-, ~ 0.790)	–	(0.4427, **0.8092**)
AutoInt	–	–	–	–	(0.4434, 0.8083)	**(0.4420, 0.8101)**
DCN-V2	–	–	–	–	–	**(0.4406, 0.8115)**
DNN	–	(0.4428, -)	(0.4491, 0.7993)	–	–	**(0.4421, 0.8098)**

嵌入大小为1。在这种情况下，所有计算都是可互换的。因此，我们采用符号以及公式9的结果，即交叉网络x^l的第l层中的第i个元素具有以下公式：

$$\mathrm{x}_i^l = \sum_{p=2}^{l+1} \sum_{I \in S_p^i} \sum_{J \in C_l^{p-1}} g(I, J; \mathrm{x}, W) + x_i \tag{10}$$

为了便于证明并简化最终公式，我们假设l层交叉网络的最终logit为$1^\top \mathrm{x}^l$，则

$$\begin{aligned}
1^\top \mathrm{x}^l &= \sum_{i=1}^{d} \sum_{p=2}^{l+1} \sum_{I \in S_p^i} \sum_{J \in C_l^{p-1}} x_{i_1} \odot \left(w_{i_1 i_2}^{(j_1)} x_{i_2} \odot \ldots \odot \left(w_{i_k i_{k+1}}^{(j_k)} x_{i_{l+1}} \right) \right) + \sum_{i=1}^{d} x_i \\
&= \sum_{p=2}^{l+1} \sum_{I \in S_p} \sum_{J \in C_l^{p-1}} w_{i_1 i_2}^{(j_1)} \ldots w_{i_k i_{k+1}}^{(j_k)} x_{i_1} x_{i_2} \ldots x_{i_{l+1}} + \sum_{i=1}^{d} x_i \\
&= \sum_{p=2}^{l+1} \sum_{|\boldsymbol{\alpha}|=p} \sum_{J \in C_l^{p-1}} \sum_{\mathrm{i} \in P_{\boldsymbol{\alpha}}} \prod_{k=1}^{|\boldsymbol{\alpha}|-1} w_{i_k i_{k+1}}^{(j_k)} x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_d^{\alpha_d} + \sum_{i=1}^{d} x_i \\
&= \sum_{\boldsymbol{\alpha}} \sum_{\mathrm{j} \in C_l^{|\boldsymbol{\alpha}|-1}} \sum_{\mathrm{i} \in P_{\boldsymbol{\alpha}}} \prod_{k=1}^{|\boldsymbol{\alpha}|-1} w_{i_k i_{k+1}}^{(j_k)} x_1^{\alpha_1} x_2^{\alpha_2} \cdots x_d^{\alpha_d} + \sum_{i=1}^{d} x_i
\end{aligned}$$

其中 $P_{\boldsymbol{\alpha}}$ 是 $(\underbrace{1 \cdots 1}_{\alpha_1 \text{ times}} \cdots \underbrace{d \cdots d}_{\alpha_d \text{ times}})$ 的所有排列的集合，$C_l^{|\boldsymbol{\alpha}|-1}$ 是一个集合，表示从整数 $\{1, \cdots, l\}$ 中一次选择 $|\boldsymbol{\alpha}| - 1$ 个索引的组合，具体来说，

$$C_l^{|\boldsymbol{\alpha}|-1} := \left\{ \mathrm{y} \in [l]^{|\boldsymbol{\alpha}|-1} \,\middle|\, (y_i \neq y_j) \wedge (y_{j_1} > y_{j_2} > \ldots > y_{j_{|\boldsymbol{\alpha}|-1}}) \right\}.$$

第二个等式将第一个和第三个求和合并为一个在新的集合 S_p^c 上求和的单一表达式：$= [c]^p$。第三个等式使用多重索引 $\boldsymbol{\alpha}$ 重新表示了交叉项（单项式），并相应地修改了权重 w 的索引。最后一个等式合并了前两个求和。证毕。□